SAMBHRAM INSTITUTE OF TECHNOLOGY

M. S. Palya, Bengaluru – 560097

DEPARTMENT OF CSE(CYBER SECURITY)

LAB MANUAL

For

Object Oriented Programming with JAVA

SUB CODE: BCS306A

Version: 1

Prepared by
Prof. B. Ganga

Reviewed by
Dr. Sanjeetha. R

SAMBHRAM INSTITUTE OF TECHNOLOGY

M. S. Palya, Bengaluru – 560097

DEPARTMENT OF CSE(CYBER SECURITY)

Review of Lab Manual

Subject: BCS306A Object Oriented Programming with JAVA

Manual Prepared by : Prof. B. Ganga

Reviewed by : Dr. Sanjeetha. R

It is certified that Lab manual for BCS306A Object Oriented Programming with JAVA of III Semester B.E., CSE (Cyber Security), Sambhram Insitute of Technology affiliated to Visvesvaraya Technological University, Belagavi, is reviewed and found to be correct and complete.

Signatures

(Prepared by) **(Reviewer)** **HOD - Dept. CSE(CY)**

PROGRAM OUTCOMES

Engineering Graduates will able to:

Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

Problem analysis: Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

Project management and finance: Demonstrate knowledge and understanding of the Engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

Life -long learning: Recognize the need for and have the preparation and ability to engage in independent and life -long learning in the broadest context of technological change.

SAMBHRAM INSTITUTE OF TECHNOLOGY

M. S. Palya, Bengaluru – 560097

INSTITUTE VISION AND MISSION

VISION

The future is embodied in the present generation. Professional education combined with practical exposure to create values in the students who are the future of our nation.

MISSION

Work oriented education combined with ethical values, character building in context of today`s millennium.

SAMBHRAM INSTITUTE OF TECHNOLOGY

M. S. Palya, Bengaluru – 560097

DEPARTMENT OF CSE(CYBER SECURITY)

DEPARTMENT VISION AND MISSION

VISION

The Cybersecurity Department aims to empower current students with hands-on skills and ethical values, preparing them to protect our nation's digital realm with expertise and integrity.

MISSION

- Fostering skilled cybersecurity professionals with ethics and character for the digital age.
- Empowering students with practical cybersecurity knowledge and strong values.
- Preparing graduates to secure the digital world through work-oriented education and ethical grounding

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO 1: Graduates will possess strong cybersecurity expertise, excelling in Computer Science & Engineering roles and contributing to a safer online environment.

PEO 2: Graduates will adeptly address cyber challenges and continuously update their skills in evolving Computer Science-related technologies.

PEO 3: Graduates will be trained to become collaborative leaders, leading cybersecurity initiatives that align with organizational objectives and showcase expertise across diverse fields. check grammar

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO 1: Understand the foundational principles of basic sciences and hardware to enable the hands-on application of theoretical knowledge

PSO 2: Develop proficiency in utilizing a wide array of programming languages to design, code, and debug efficient and innovative software solutions for diverse applications.

PSO 3: Create secure system architectures, evaluate vulnerabilities, conduct penetration tests, and devise incident response strategies, guaranteeing robust defense against cyber threats and seamless operational flow.

COURSE LEARNING OBJECTIVES

- To learn primitive constructs JAVA programming language.
- To understand Object Oriented Programming Features of JAVA.
- To gain knowledge on: packages, multithreaded programing and exceptions. Illustrate the process of structuring the data using lists, tuples

COURSE OUTCOMES

CO1: Demonstrate proficiency in writing simple programs involving branching and looping structures.

CO2: Design a class involving data members and methods for the given scenario.

CO3: Apply the concepts of inheritance and interfaces in solving real world problems.

CO4: Use the concept of packages and exception handling in solving complex problem

CO5: Apply concepts of multithreading, autoboxing and enumerations in program development

Assessment Details (both CIE and SEE)

The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks (18 out of 50 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/ course if the student secures a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together.

CIE for the theory component of the IPCC (maximum marks 50)

- IPCC means practical portion integrated with the theory of the course.
- CIE marks for the theory component are **25 marks** and that for the practical component is **25 marks**.
- 25 marks for the theory component are split into **15 marks** for two Internal Assessment Tests (Two Tests, each of 15 Marks with 01-hour duration, are to be conducted) and **10 marks** for other assessment methods mentioned in 22OB4.2. The first test at the end of 40-50% coverage of the syllabus and the second test after covering 85-90% of the syllabus.
- Scaled-down marks of the sum of two tests and other assessment methods will be CIE marks for the theory component of IPCC (that is for **25 marks)**.
- The student has to secure 40% of 25 marks to qualify in the CIE of the theory component of IPCC. **CIE for the practical component of the IPCC**
- **15 marks** for the conduction of the experiment and preparation of laboratory record, and **10 marks** for the test to be conducted after the completion of all the laboratory sessions.
- On completion of every experiment/program in the laboratory, the students shall be evaluated including viva-voce and marks shall be awarded on the same day.
- The CIE marks awarded in the case of the Practical component shall be based on the continuous evaluation of the laboratory report. Each experiment report can be evaluated for 10 marks. Marks of all experiments' write-ups are added and scaled down to **15 marks**.
- The laboratory test **(duration 02/03 hours)** after completion of all the experiments shall be conducted for 50 marks and scaled down to **10 marks.**
- Scaled-down marks of write-up evaluations and tests added will be CIE marks for the laboratory component of IPCC for **25 marks**.
- The student has to secure 40% of 25 marks to qualify in the CIE of the practical component of the IPCC.

SEE for IPCC

Theory SEE will be conducted by University as per the scheduled timetable, with common question papers for the course (**duration 03 hours**)

1. The question paper will have ten questions. Each question is set for 20 marks.
2. There will be 2 questions from each module. Each of the two questions under a module (with a maximum of 3 sub-questions), **should have a mix of topics** under that module.
3. The students have to answer 5 full questions, selecting one full question from each module.
4. Marks scored by the student shall be proportionally scaled down to 50 Marks

The theory portion of the IPCC shall be for both CIE and SEE, whereas the practical portion will have a CIE component only. Questions mentioned in the SEE paper may include questions from the practical component.

12	Develop a program to create a class MyThread in this class a constructor, call the base class constructor, using super and start the thread. The run method of the class starts after this. It can be observed that both main thread and created child thread are executed concurrently.	22

1. Develop a JAVA program to add TWO matrices of suitable order N (The value of N should be read from command line arguments).

Save Filename as: MatrixAddition.java

Solution:-

```
public class MatrixAddition
{
  public static void main (String[] args)
  {
    int n = Integer.parseInt (args[0]);
    int[][] matrix1 = new int[n][n];
    int[][] matrix2 = new int[n][n];
    int[][] sum = new int[n][n];
    // Initialize matrices with some values, for example, i+j
    for (int i = 0; i < n; i++)
      {
        for (int j = 0; j < n; j++)
          {
            matrix1[i][j] = i + j;
            matrix2[i][j] = i + j;
          }
      }
    // Add the matrices
    for (int i = 0; i < n; i++)
      {
        for (int j = 0; j < n; j++)
          {
            sum[i][j] = matrix1[i][j] + matrix2[i][j];
          }
      }
    // Print the result
    System.out.println ("Sum of matrices is: ");
    for (int i = 0; i < n; i++)
      {
        for (int j = 0; j < n; j++)
          {
            System.out.print (sum[i][j] + " ");
          }
        System.out.println ();
      }
  }
}
```

Compile as: javacMatrixAddition.java

Run as: java MatrixAddition 3

Output:

```
Sum of matrices is:
0 2 4
2 4 6
6 8 7
```

2. Develop a stack class to hold a maximum of 10 integers with suitable methods. Develop a JAVA main method to illustrate Stack operations

Save Filename as:StackMain.java

Solution:-

```
import java.util.Scanner;
class Stack
{
  private int maxSize = 10;
  private int top;
  private int[] stackArray;
  public Stack ()
  {
    stackArray = new int[maxSize];
    top = -1;
  }

  public void
  push (int value)
  {
    if (top == maxSize - 1)
      {
  System.out.println("Stack is full. Unable to
  push " + value);
  return;
      }
    stackArray[++top] = value;
  }
  public void
  pop ()
  {
    if (top == -1)
      {
        System.out.println ("Stack is empty");
        return;
      }
    System.out.println ("Popped " + stackArray[top--] + "from the
    stack");
  }
  public void
  display ()
  {
    if (top == -1)
      {
        System.out.println ("Stack is empty");
        return;
      }
    System.out.print ("Stack: ");
    for (int i = 0; i <= top; i++)
      {
```

```
System.out.print (stackArray[i] + " ");
        }
      System.out.println ();
    }
 }
 public class StackMain
 {
   public static void main (String[] args)
   {
     Stack stack = new Stack ();
     Scanner scanner = new Scanner (System.in);
     while (true)
       {
         System.out.println ("Choose an option:");
         System.out.println ("1) Push");
         System.out.println ("2) Pop");
         System.out.println ("3) Display");
         System.out.println ("4) Exit");
         int option = scanner.nextInt ();
         switch (option)
           {
           case 1:
             System.out.println ("Enter a number to push:");
             int num = scanner.nextInt ();
             stack.push (num);
             break;

           case 2:
             stack.pop ();
             break;

           case 3:
             stack.display ();
             break;

           case 4:
             scanner.close ();
             return;

           default:
                    System.out.println("Invalid option.Please
                    choose again.");
           }
       }
   }
 }
```

Compile As: javacStackMain.java
Run As: java StackMain

Output:

```
Choose an option:
1) Push
2) Pop
3) Display
4) Exit

Enter a number to push:
10
Choose an option:
1) Push
2) Pop
3) Display
4) Exit

Enter a number to push:
20
Choose an option:
1) Push
2) Pop
3) Display
4) Exit

Enter a number to push:
30
Choose an option:
1) Push
2) Pop
3) Display
4) Exit

Stack: 10 20 30
Choose an option:
1) Push
2) Pop
3) Display
4) Exit

Popped 30 from the stack
Choose an option:
1) Push
2) Pop
3) Display
4) Exit

Stack: 10 20
```

```
Choose an option:
1) Push
2) Pop
3) Display
4) Exit
```

3. A class called Employee, which models an employee with an ID, name and salary, is designed as shown in the following class diagram. The method raiseSalary (percent) increases the salary by the given percentage. Develop the Employee class and suitable main method for demonstration.
`Save Filename as as: EmployeeMain.java`

Solution:-

```
import java.util.Scanner;
class Employee
{
  private int id;
  private String name;
  private double salary;
  public Employee (int id, String name, double salary)
  {
    this.id = id;
    this.name = name;
    this.salary = salary;
  }
  public int
  getId ()
  {
    return id;
  }
  public String
  getName ()
  {
    return name;
  }
  public double
  getSalary ()
  {
    return salary;
  }
  public void
  raiseSalary (double percent)
  {
    salary += salary * percent / 100.0;
  }
}
public class EmployeeMain
{
  public static void main (String[] args)
  {
    Scanner scanner = new Scanner (System.in);
    System.out.println ("Enter Employee ID:");
    int id = scanner.nextInt ();
    System.out.println ("Enter Employee Name:");
    scanner.nextLine (); // Consume newline left-over
    String name = scanner.nextLine ();
    System.out.println ("Enter Employee Salary:");
```

```
    double salary = scanner.nextDouble ();
    Employee emp = new Employee (id, name, salary);
    System.out.println ("Employee ID: " + emp.getId ());
    System.out.println ("Employee Name: " + emp.getName ());
    System.out.println ("Employee Salary: " + emp.getSalary ());
    System.out.println ("Enter raise percentage:");
    double percent = scanner.nextDouble ();
    emp.raiseSalary (percent);
    System.out.println ("Employee Salary after raise: " +
emp.getSalary ());
    scanner.close ();
  }
}
```

Compile As: javacEmployeeMain.java
Run As: java EmployeeMain

Output:

```
Enter Employee ID:
1
Enter Employee Name:
ABC
Enter Employee Salary:
20000
Employee ID: 1
Employee Name: ABC
Employee Salary: 20000.0
Enter raise percentage:
15
Employee Salary after raise: 23000.0
```

4. A class called MyPoint, which models a 2D point with x and y coordinates, is designed as follows:Two instance variables x (int) and y (int).

- **A default (or "no-arg") constructor that construct a point at the default location of (0, 0).**
- **A overloaded constructor that constructs a point with the given x and y coordinates.**
- **A method setXY() to set both x and y.**
- **A method getXY() which returns the x and y in a 2-element int array.**
- **A toString() method that returns a string description of the instance in the format "(x, y)".**
- **A method called distance(int x, int y) that returns the distance from this point to another point at the given (x, y) coordinates**
- **An overloaded distance(MyPoint another) that returns the distance from this point to the given MyPoint instance (called another)**
- **Another overloaded distance() method that returns the distance from this point to the origin (0,0) Develop the code for the class MyPoint. Also develop a JAVA program (called TestMyPoint) to test all the methods defined in the class.**

Save Filename As: TestMyPoint.java

Solution:-

```
class MyPoint
{
  private int x;
  private int y;

  // Default Constructor
  public MyPoint () { this(0, 0); }

  // Overloaded Constructor
  public MyPoint (int x, int y)
  {
    this.x = x;
    this.y = y;
  }

  // Setters
  public void
  setXY (int x, int y)
  {
    this.x = x;
    this.y = y;
  }

  // Getters
  public int[] getXY ()
  {
    int[] coordinates = { x, y };
    return coordinates;
  }

  // Calculate distance to another point (x, y)
  public double
```

```
  distance (int x, int y)
  {
    return Math.sqrt (Math.pow (this.x - x, 2) + Math.pow (this.y -
y, 2));
  }
```

The formula for the given Java `distance` method is the Euclidean distance formula in two dimensions, which calculates the distance between two points (x_1, y_1) and (x_2, y_2). Here's the formula:

$$\text{distance} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

In the context of your method, if `this.x` and `this.y` represent the coordinates (x_1, y_1) of the object, and x and y are the coordinates (x_2, y_2) of the second point, the formula is:

$$\text{distance} = \sqrt{(\text{this.x} - x)^2 + (\text{this.y} - y)^2}$$

```
  // Calculate distance to another MyPoint object
  public double
  distance (MyPoint another)
  {
    return Math.sqrt (Math.pow (this.x - another.x, 2)
                      + Math.pow (this.y - another.y, 2));
  }
```

$$\text{distance} = \sqrt{(\text{this.x} - \text{another.x})^2 + (\text{this.y} - \text{another.y})^2}$$

```
  // Calculate distance to the origin (0,0)
  public double
  distance ()
  {
    return Math.sqrt (Math.pow (this.x, 2) + Math.pow (this.y, 2));
  }
```

The formula for calculating the distance from a point (x, y) to the origin $(0, 0)$ is derived from the Euclidean distance formula. Here, `this.x` and `this.y` represent the coordinates (x, y) of the point.

The formula is:

$$\text{distance} = \sqrt{x^2 + y^2}$$

In this context:

$$\text{distance} = \sqrt{(\text{this.x})^2 + (\text{this.y})^2}$$

```
  // String representation of the point
```

```
    @Override
    public String
    toString ()
    {
      return "(" + x + ", " + y + ")";
    }
  }
```

sion conca

"(3, 4)".

```
  public class TestMyPoint
  {
    public static void main (String[] args)
    {
      MyPoint point1 = new MyPoint ();      // Default constructor
      MyPoint point2 = new MyPoint (3, 4); // Overloaded
      constructor point1.setXY (5, 6);      // Set x and y

      int[] coordinates = point2.getXY (); // Get x and y

      System.out.println ("Point 1: " + point1);
      System.out.println ("Point 2: " + point2);
      System.out.println ("Point 2 coordinates: (" + coordinates[0] +
  ", "
                          + coordinates[1] + ")");
      System.out.println ("Distance from Point 1 to (5, 6): "
                          + point2.distance (point1));
        System.out.println("Distance from Point 2 to Point 1: " +
        System.out.println("Distance from Point 2 to origin: " +
```

```
        point2.distance());
    }
}
```

Compile As: javacTestMyPoint.java
Run As: java TestMyPoint

Output:

```
Point 1: (5, 6)
Point 2: (3, 4)
Point 2 coordinates: (3, 4)
Distance from Point 1 to (5, 6): 0.0
Distance from Point 2 to Point 1: 2.8284271247461903
Distance from Point 2 to origin: 5.0
```

5. Develop a JAVA program to create a class named shape. Create three sub classes namely: circle, triangle and square, each class has two member functions named draw () and erase (). Demonstrate polymorphism concepts by developing suitable methods, defining member data and main program

Save Filename as: ShapeMain.java

Solution:-

```
class Shape
{
  public void
  draw ()
  {
    System.out.println ("Drawing a shape");
  }

  public void
  erase ()
  {
    System.out.println ("Erasing a shape");
  }
}
class Circle extends Shape
{
  @Override
  public void
  draw ()
  {
    System.out.println ("Drawing a circle");
  }

  @Override
  public void
  erase ()
  {
    System.out.println ("Erasing a circle");
  }
}
class Triangle extends Shape
{
  @Override
  public void
  draw ()
  {
    System.out.println ("Drawing a triangle");
  }

  @Override
  public void
  erase ()
  {
```

```
    System.out.println ("Erasing a triangle");
  }
}

class Square extends Shape
{
  @Override
  public void
  draw ()
  {
    System.out.println ("Drawing a square");
  }

  @Override
  public void
  erase ()
  {
    System.out.println ("Erasing a square");
  }
}
public class ShapeMain
{
  public static void main (String[] args)
  {
    Shape[] shapes = new Shape[3];
    shapes[0] = new Circle ();
    shapes[1] = new Triangle ();
    shapes[2] = new Square ();
    for (Shape shape : shapes)
      {
        shape.draw ();
        shape.erase ();
        System.out.println (); // Add a line break for clarity
      }
  }
}
```

Compile As: javacShapeMain.java
Run As: java ShapeMain

Output:

```
Drawing a circle
Erasing a circle

Drawing a triangle
Erasing a triangle

Drawing a square
Erasing a square
```

6. Develop a JAVA program to create an abstract class Shape with abstract methods calculateArea() and calculatePerimeter(). Create subclasses Circle and Triangle that extend the Shape class and implement the respective methods to calculate the area and perimeter of each shape.

Save Filename as: Shape_peri_area_Main.java

Solution:-

```
abstract class Shape
{
  // Abstract methods to calculate area and perimeter
  public abstract double calculateArea ();
  public abstract double calculatePerimeter ();
}
class Circle extends Shape
{
  private double radius;

  // Constructor
  public Circle (double radius) { this.radius = radius; }

  @Override
  public double
  calculateArea ()
  {
    return Math.PI * radius * radius;
  }

  @Override
  public double
  calculatePerimeter ()
  {
    return 2 * Math.PI * radius;
  }
}
class Triangle extends Shape
{
  private double side1;
  private double side2;
  private double side3;

  // Constructor
  public Triangle (double side1, double side2, double side3)
  {
    this.side1 = side1;
    this.side2 = side2;
    this.side3 = side3;
  }

  @Override
  public double
```

```
  calculateArea ()
  {
    double s = (side1 + side2 + side3) / 2;
    return Math.sqrt (s * (s - side1) * (s - side2) * (s - side3));
  }

  @Override
  public double
  calculatePerimeter ()
  {
    return side1 + side2 + side3;
  }
}
public class Shape_peri_area_Main
{
  public static void main (String[] args)
  {
    Circle circle = new Circle (5.0);
    Triangle triangle = new Triangle (3.0, 4.0, 5.0);

    System.out.println ("Circle:");
    System.out.println ("Area: " + circle.calculateArea ());
    System.out.println ("Perimeter: " + circle.calculatePerimeter
());

    System.out.println ();

    System.out.println ("Triangle:");
    System.out.println ("Area: " + triangle.calculateArea ());
    System.out.println ("Perimeter: " + triangle.calculatePerimeter
());
  }
}
```

Compile As: javacShape_peri_area_Main.java
Run As: java Shape_peri_area_Main

Output:

```
Circle:
Area: 78.53981633974483
Perimeter: 31.41592653589793

Triangle:
Area: 6.0
Perimeter: 12.0
```

7. D evelop a JAVA program to create an interface Resizable with methods resizeWidth(int width) and resizeHeight(int height) that allow an object to be resized. Create a class Rectangle that implements the Resizable interface and implements the resize methods.

Save Filename as: rect_resize.java

Solution:-

```
interface Resizable
{
  void resizeWidth (int width);
  void resizeHeight (int height);
}

class Rectangle implements Resizable
{
  private int width;
  private int height;

  // Constructor
  public Rectangle (int width, int height)
  {
    this.width = width;
    this.height = height;
  }

  @Override
  public void
  resizeWidth (int width)
  {
    this.width = width;
  }

  @Override
  public void
  resizeHeight (int height)
  {
    this.height = height;
  }

  // Additional method to get the dimensions
  public void
  getDimensions ()
  {
    System.out.println ("Width: " + width + ", Height: " + height);
  }
}

public class rect_resize
{
  public static void main (String[] args)
  {
```

```
    Rectangle rectangle = new Rectangle (5, 7);

    System.out.println ("Original Dimensions:");
    rectangle.getDimensions ();

    rectangle.resizeWidth (8);
    rectangle.resizeHeight (10);

    System.out.println ("\nResized Dimensions:");
    rectangle.getDimensions ();
  }
}
```

Compile As: javacrect_resize.java
Run As: java rect_resize

Output:

```
Original Dimensions:
Width: 5, Height: 7

Resized Dimensions:
Width: 8, Height: 10
```

8. Develop a JAVA program to create an outer class with a function display. Create another class inside the outer class named inner with a function called display and call the two functions in the main class.

Save Filename As: InnerOuter.java

Solution:-

```
class Outer
{
    void display()
    {
      System.out.println("Outer display");
    }

    class Inner
    {
      void display()
       {
          System.out.println("Inner display");
       }
    }
}

public class InnerOuter
{
    public static void main(String[] args)
     {
        Outer outer = new Outer();
        Outer.Inner inner = outer.newInner();
        outer.display(); // Calls the outer class display()
        function
        inner.display(); // Calls the inner class display()
        function
     }
}
```

Compile As: javacInnerOuter.java
Run As: java InnerOuter

Output:

```
Outer display
Inner display
```

9. Develop a JAVA program to raise a custom exception (user defined exception) for DivisionByZero using try, catch, throw and finally

Save Filename As: CustomException.java

Solution:-

```
// Custom Exception Class
class DivisionByZeroException extends Exception
{
  public DivisionByZeroException (String message) { super
(message); }
}
public class CustomException
{
  public static void main (String[] args)
  {
    int dividend = 10;
    int divisor = 0;
    try
      {
        if (divisor == 0)
          {
                throw new DivisionByZeroException("Cannot divide
                by zero");
          }
        int result = dividend / divisor;
        System.out.println ("Result: " + result);
      }
    catch (DivisionByZeroException e)
      {
        System.out.println ("Exception: " + e.getMessage ());
      }
    finally
      {
        System.out.println ("Finally block executed");
      }
  }
}
```

Compile As: javacCustomException.java
Run As: java CustomException

Output:

```
Exception: Cannot divide by zero
Finally block executed
```

10. Develop a JAVA program to create a package named mypack and import & implement it in a suitable class

Step 1: Create a Package

Create a directory named mypack (or any name you prefer) on your file system. Inside the mypack directory, create a Java file named MyClass.java.

Step 2: Define the Class in the Package. Open MyClass.java and add the following code:

```
package mypack;
public class MyClass
{
  public void
  display ()
  {
    System.out.println("This is a method from the mypack
    package.");
  }
}
```

Step 3: Create a Main Program

Create a new Java file (outside the mypack directory) named MainProgram.java and add the following code:

```
import mypack.MyClass;

public class MainProgram
{
  public static void main (String[] args)
  {
    MyClass obj = new MyClass ();
    obj.display ();
  }
}
```

Step 4: Compile and Run

Open a terminal or command prompt and navigate to the directory containing both the mypack directory and MainProgram.java.

Compile both files using the following command:

```
javacmypack/MyClass.java MainProgram.java
```

Run the program with the command:

```
java MainProgram
```

Output:

```
This is a method from the mypack package.
```

11. Write a program to illustrate creation of threads using runnable class. (start method start each of the newly created thread. Inside the run method there is sleep() for suspend the thread for 500 milliseconds)

Save Filename as: ThreadMain.java

Solution:-

```
class MyRunnable implements Runnable
 {
    @Override
    public void run()
    {
        Try
         {
          System.out.println("Thread " +
          Thread.currentThread().getId() + " is running.");
          Thread.sleep(500);
        }
        catch (InterruptedException e)
         {
          System.out.println("Thread interrupted: " +
          e.getMessage());
        }
    }
}

public class ThreadMain
 {
    public static void main(String[] args)
    {
       int numThreads = 5;
       for (int i = 0; i<numThreads; i++)
        {
          Thread thread = new Thread(new MyRunnable());
          thread.start();
        }
    }
}
```

Compile As: javacThreadMain.java
Run As: java ThreadMain

Output:

```
Thread 10 is running.
Thread 12 is running.
Thread 14 is running.
Thread 13 is running.
Thread 11 is running.
```

12. Develop a program to create a class MyThread in this class a constructor, call the base class constructor, using super and start the thread. The run method of the class starts after this. It can be observed that both main thread and created child thread are executed concurrently.

Save Filename as: ThreadConstructor.java

Solution:-

```
class MyThread extends Thread
{
  MyThread (String name)
  {
    super (name); // Call the base class constructor with thread
    name start (); // Start the thread
  }

  public void
  run ()
  {
    try
      {
        for (int i = 5; i > 0; i--)
          {
            System.out.println (getName () + ": " + i);
            Thread.sleep (1000);
          }
      }
    catch (InterruptedException e)
      {
        System.out.println (getName () + " interrupted.");
      }
    System.out.println (getName () + " exiting.");
  }
}

public class ThreadConstructor
{
  public static void main (String[] args)
  {
    new MyThread ("Child Thread");

    try
      {
        for (int i = 5; i > 0; i--)
          {
            System.out.println ("Main Thread: " + i);
            Thread.sleep (2000);
          }
      }

    catch (InterruptedException e)
      {
```

```
        System.out.println ("Main Thread interrupted.");
      }
    System.out.println ("Main Thread exiting.");
  }
}
```

Compile As: javacThreadConstructor.java
Run As: java ThreadConstructor

Output:

```
Main Thread: 5
Child Thread: 5
Child Thread: 4
Main Thread: 4
Child Thread: 3
Child Thread: 2
Main Thread: 3
Child Thread: 1
Child Thread exiting.
Main Thread: 2
Main Thread: 1
Main Thread exiting.
```